

Jour 2 : Zéro dépendance à l'OS

D'abord voici brièvement ce qui se passe quand on écrit un programme Rust

```
ton code
  ↓
bibliothèque standard (std)
  ↓
appels système (syscalls)
  ↓
le noyau OS (Linux, Windows...)
  ↓
le hardware
```

La std de Rust te donne accès à des fonctions basiques telles que :

- `println!` → écriture console
- `Vec`, `String` → allocation mémoire dynamique
- `File` → système de fichiers
- `Thread` → threads
- `TcpStream` → réseau

Tout ça suppose qu'un OS existe en dessous. Dans le projet, on développe une OS de à partir de zéro. Donc std est impossible à utiliser.

```
#![no_std] // cette ligne dit à Rust : "n'importe pas std"
```

Créer ton premier binaire `no_std`

Focus Sécurité : Pourquoi ça réduit les vulnérabilités ?

- Surface d'attaque réduite

```
Avec std :
ton code → std (100 000+ lignes) → OS → hardware
          ↑
          plein de vecteurs d'attaque ici

Sans std :
ton code → core (minimal) → hardware
```

- Pas de syscalls = pas d'interface d'attaque OS
- pas de heap exploits : En `no_std`, **il n'y a pas de heap du tout par défaut**. Pas de `malloc`, pas de `free`. Les attaques comme (**Use-after-free**, **Heap overflow**, **Double free**) deviennent impossibles.
- Pas de code caché qui tourne en arrière-plan, pas de runtime. Tu sais exactement ce qui s'exécute.

```
#![no_std]    // pas de bibliothèque standard
#![no_main]  // pas de fonction main() classique

use core::panic::PanicInfo;

// Point d'entrée de ton OS
// "no_mangle" = garde le nom "_start" tel quel pour le linker
// "extern C" = utilise la convention d'appel du C
#[no_mangle]
pub extern "C" fn _start() -> ! {
    // Le "-> !" signifie que cette fonction ne retourne JAMAIS
    // Normal : un OS ne "termine" pas
    loop {}
}

// En no_std, tu DOIS définir ce qui se passe en cas de panic
// Parce que std le faisait pour toi avant
#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    // Pour l'instant : boucle infinie
    // Plus tard : tu afficheras l'erreur sur le port série
    loop {}
}
```

Ensuite on écrit les informations pour le linker :

```
mkdir .cargo
nano .cargo/config.toml
```

Et voici ces instructions

```
[build]
target = "x86_64-unknown-none" # toujours compiler en bare-metal
```

```
[target.x86_64-unknown-none]
rustflags = [
  "-C", "link-arg=-nostartfiles", # pas de fichiers de démarrage C
]
```

Et enfin on compile :

```
(root@kali)-[/home/kali/Day2/OS_Day2]
# nano -l src/main.rs

(rroot@kali)-[/home/kali/Day2/OS_Day2]
#

(rroot@kali)-[/home/kali/Day2/OS_Day2]
# cargo build -Z build-std=core --target x86_64-unknown-none
Compiling OS_Day2 v0.1.0 (/home/kali/Day2/OS_Day2)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.49s

(rroot@kali)-[/home/kali/Day2/OS_Day2]
# qemu-system-x86_64 \
  -kernel target/x86_64-unknown-none/debug/OS_Day2 \
  -serial stdio \
  -display none
qemu-system-x86_64: Error loading uncompressed kernel without PVH ELF Note
```

Evidemment on ne peut pas faire le boot sur une quelconque hyperviseur (comme qemu) et cela pour deux raisons :

1. Pas de bootloader qui va dire au BIOS quel code charger
2. Comme vous pouvez le voir sur l'image, on doit avoir un binaire en format ELF ou image disque

Les avantages :

```
Binaire normal :
[code] + [std 100k lignes] + [runtime C] + [syscalls OS]
→ plein de code que tu ne contrôles pas

Notre binaire no_std :
[code] + [core minimal]
→ Nous contrôlons TOUT ce qui s'exécute
```